Albert's Task Notes for Gagiteck (March – April)

Clone repos and confirm local dev environment setup

Backend:

- Cloned the repo and completed local setup.
- Created and verified Python environment and dependencies
- Configured .env.development
- Started required local service and launched the FastAPI backend on port 8000
- Confirmed local API docs & health endpoints are reachable (/docs, /openapi.json, /health).

Frontend:

- Cloned the repo and completed local setup.
- Installed Node dependencies and configured .env.development.
- Confirmed the frontend runs on port 3000, local frontend configuration uses NEXT_PUBLIC_API_URL=http://localhost:8000

Note to self: Backend README mentions seeded test login credentials, but the current auth router appears to return HTTP 410 for password login/signup. I believe it directs users to OIDC/SSO instead unless I missed something?

Evaluate workflow & branching strategy:

- Both backend and frontend use a PR-based development workflow with GitHub Actions enforcing branch rules. The permitted merge flow is structured as:
  - feature/bugfix/release/docs/chore/refactor/perf/test/ci branches → `develop`
  - `develop` or `main` → `staging`
  - `staging` or `hotfix/*` → `main`
- The repos also enforce naming conventions for most work branches using Jira-linked formats; PR titles are expected to include the issue key from the branch name. Live Jira validation can also occur when Jira credentials are configured.
- In effect, the team is provided a defined branching strategy rather than an ad hoc one.

Tradeoffs

- Branch rules and naming conventions are somewhat strict, which risks possible friction for new contributors or when making very small changes.
- There also appears to be at least one onboarding/documentation mismatch on the backend side, where the README still references seeded password-login credentials while the current auth router appears to direct users to OIDC/SSO instead.

Evaluate CI/CD readiness:

Overall assessment:

Frontend & backend CI/CD appear to have structured CI/CD pipelines with substantive evidence of readiness. There are a few documentation and workflow-consistency tidbits worth noting:

- Backend and frontend staging deploy workflows are slightly inconsistent in their defensive branch guards.
- Dev deploys in both repos appear to have weaker sanity-test coverage than staging/production (relevant URL input is blank).
- Staging workflows appear to use a cluster (`gagiteck-dev-cluster`) named as dev, perhaps as intentional shared infrastructure, but may be ambiguous from an environment-labeling perspective because staging otherwise appears logically separated from dev at service, branch, secret, and URL levels.

Backend CI readiness:

- Backend repo has a GitHub Actions CI pipeline that runs on pushes and pull requests to `develop`, `staging`, and `main`.
    - It includes linting, type checking, automated tests with Postgres/Redis service containers, dependency vulnerability scanning, and Docker image build validation.

Backend CD readiness:

- Backend deployment is environment-specific and CI-gated: successful CI on `develop`, `staging`, and `main` triggers deploy workflows for `dev`, `staging`, and `production` respectively.
    - (evidence: `workflow_run` of `["CI"]` on corresponding environment branch, requiring successful CI completion; deploy jobs add an explicit matching `head_branch` guard)
- The reusable deploy workflow includes:
    - AWS credential setup (auth with AWS for permission to read secrets, read ECS task definitions, push container images, update services)
    - preflight OIDC/auth config checks
    - **runtime secret validation (`Validate runtime secret payload in deploy_reusable.yml`)**
    - rendered ECS task-defintion validation
    - migration checks
    - ECS rollout / deploy (`aws-actions/amazon-ecs-deploy-task-definition@v2`)
    - sanity testing with `healthcheck_url` supplied
    - release creation for production (`create_release: true` in `deploy-production.yml`)

Frontend CI readiness:

- Frontend repo has a GitHub Actions CI pipeline that runs on pushes and pull requests to `develop`, `staging`, and `main`.
    - It includes ESLint, TypeScript type checking, Vitest test runs, a security job, and a Docker build job after the earlier checks pass.
- Workflow also installs a Rollup native binary on Linux runners when needed → suggests it explicitly handles a known platform/dependency issue.

Frontend CD readiness:

- Frontend deployment is also environment-specific and CI-gated: successful CI on `develop`, `staging`, and `main` triggers deploy workflows for `dev`, `staging`, and `production` respectively.

    - (evidence: `workflow_run` of `["CI"]` on corresponding environment branch, requiring successful CI completion; some workflows also add an explicit `head_branch` guard)
- The reusable deploy workflow includes:
    - ECS service validation (check if can resolve/use intended ECS target for deployment)
    - auth/session validation
    - Auth0 configuration checks
    - **base URL/deployed URL consistency checks**
    - sanity testing with an app URL supplied
    - release creation for production (`create_release: true` in `deploy-production.yml`)

With workflows as the source of truth on CI/CD automation, frontend & backend Makefiles and READMEs corroborate them to aid developer readiness and local reproducibility.

- Backend Makefile/README documents local setup as standardized around:
    - `make setup, docker-up, migrate, dev,` optionally `seed`
- Frontend Makefile/README documents local setup as standardized around:
    - `make setup, dev, build, test, lint`

Workflows (.github/workflows/)

Workflow	Repo	Purpose / Trigger	Main actions	Other notes
`branch-policy.yml`	Backend	Runs on pull request targeting `develop`, `staging`, `main`.	Enforces permitted branch flow (source branches for each target branch) and naming rules (Jira-linked patterns).	
`ci.yml`	Backend	Runs on push & pull request for `develop`, `staging`, `main`.	Linting, type checking, automated tests, dependency vulnerability scan, Docker build validation	
	Frontend		ESLint, TypeScript type checking, Vitest test runs, security job, Docker build validation.	
`deploy-dev.yml`	Backend	Deploys dev after successful CI workflow on `develop`.	Calls reusable backend deploy workflow with dev env settings	dev appears to skip sanity testing because `healthcheck_url` is blank.
	Frontend		Calls reusable frontend deploy workflow with dev env settings	dev appears to skip sanity testing because `app_url` is blank.
`deploy-production.yml`	Backend	Deploys production after successful CI workflow on `main`.	Calls reusable backend deploy workflow with production settings.	
	Frontend		Calls reusable frontend	Unlike backend, it does not

			deploy workflow with production settings.	appear to add the same explicit `head_branch == 'main'` guard.
`deploy-reusable.yml`	Backend	Shared deployment workflow used by backend dev/staging/production deploy workflows.	AWS credential setup, pre-deploy OIDC checks, runtime secret payload validation, rendered ECS task-definition validation, migration checks, ECS deploy/rollout, sanity test, release creation.	
	Frontend	Shared deployment workflow used by frontend dev/staging/production deploy workflows.	ECS service validation, auth/session secret checks, Auth0 config checks, expected base-URL validation, task-definition update, ECS deploy, sanity test, release creation.	
`deploy-staging.yml`	Backend	Deploys staging after successful CI workflow on `staging`	Calls reusable backend deploy workflow with staging environment settings.	Unlike backend, frontend does not appear to add the same explicit `head_branch == 'staging'` guard. Probably merely defensive but may be worth standardizing for consistency. Both point at cluster named `gagiteck-dev-cluster`
	Frontend		Calls reusable frontend deploy workflow with staging environment settings.	
`notification.yml`	Backend	Handles deployment/PR-related notifications.	Sends operational notifications such as deploy status and review-related events.	